

Most of the slides are taken from

Eric Freeman, Elisabeth Robson, Bert Bates,
Kathy Sierra: Head First Design Patterns,
O'Reilly Media 2004

Using Design Patterns together

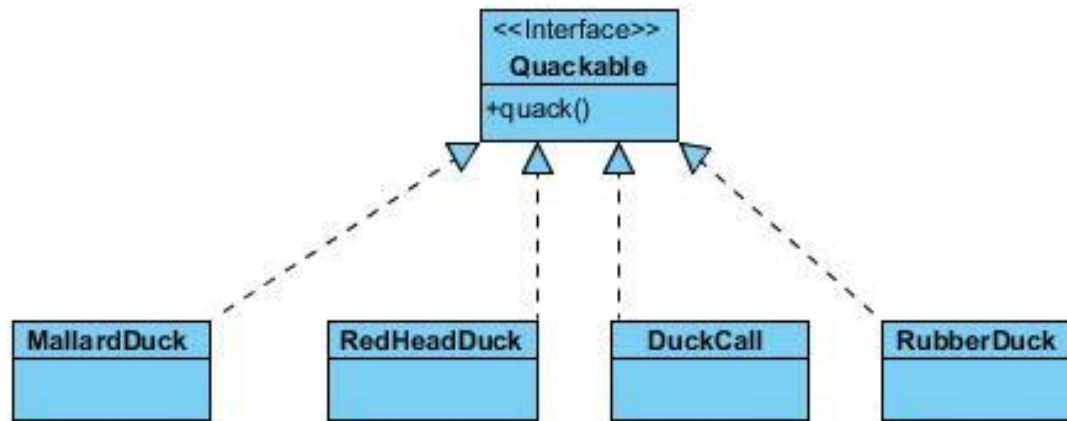
Francesco Guerra
francesco.guerra@unimore.it

Using Design Patterns together

- ▶ Design Patterns can be used together
 - ▶ Combined within the same design solution, to solve "complex" problems
 - ▶ Combined into a solution that solves a recurring or general problem
 - ▶ In this case, they are called compound patterns
 - ▶ We propose only an example of this

A complex problem

- ▶ Let us suppose to have an enterprise that creates a simulation game based on "ducks"
 - ▶ Duck can have different behaviors according with their model
 - ▶ E.g. Ducks can make "quack" if they are mallard or "squeack" if they are rubber-ducks ...
- ▶ A design based on the strategy pattern allows you to model the different behaviors



It makes quack! It makes quack! It makes kwak! It makes squeack!

The Strategy implementation

```
public interface Quackable {  
    public void quack();  
}
```

```
public class MallardDuck implements Quackable  
{  
    public void quack() {  
        System.out.println("Quack");  
    }  
}
```

```
public class RedheadDuck implements Quackable  
{  
    public void quack() {  
        System.out.println("Quack");  
    }  
}
```

```
public class DuckCall implements Quackable {  
    public void quack() {  
        System.out.println("Kwak");  
    }  
}
```

```
public class RubberDuck implements Quackable {  
    public void quack() {  
        System.out.println("Squeak");  
    }  
}
```



The Strategy implementation the simulator

```
public class DuckSimulator {  
    public static void main(String[] args) {  
        DuckSimulator simulator = new DuckSimulator();  
        simulator.simulate();  
    }  
  
    void simulate() {  
        Quackable mallardDuck = new MallardDuck();  
        Quackable redheadDuck = new RedheadDuck();  
        Quackable duckCall = new DuckCall();  
        Quackable rubberDuck = new RubberDuck();  
  
        System.out.println("\nDuck Simulator");  
  
        simulate(mallardDuck);  
        simulate(redheadDuck);  
        simulate(duckCall);  
        simulate(rubberDuck);  
    }  
  
    void simulate(Quackable duck) {  
        duck.quack();  
    }  
}
```

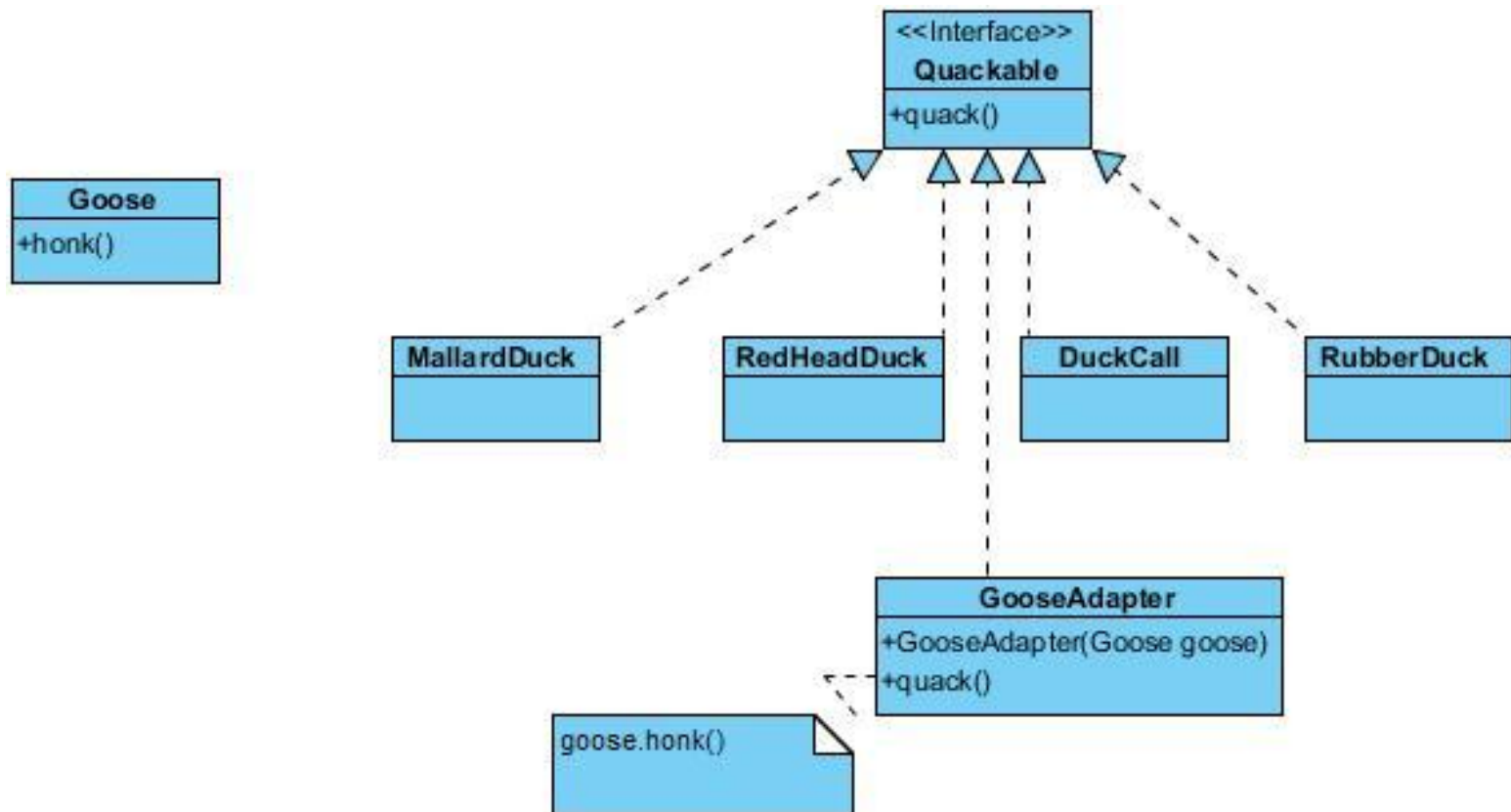
Here we overload the simulate method to simulate just one duck.

we let polymorphism do its magic: no matter what kind of Quackable gets passed in, the simulate() method asks it to quack.

```
public class Goose {  
    public void honk() {  
        System.out.println("Honk");  
    }  
}
```

- ▶ We would like to model also Geese
- ▶ What pattern would allow Geese to easily intermingle with Ducks?

A goose adapter



A goose adapter

```
public class GooseAdapter implements Quackable {  
    Goose goose;  
  
    public GooseAdapter(Goose goose) {  
        this.goose = goose;  
    }  
  
    public void quack() {  
        goose.honk();  
    }  
}
```

- ▶ An Adapter implements the target interface, which in this case is Quackable.
- ▶ The constructor takes the goose we are going to adapt.
- ▶ When quack is called, the call is delegated to the goose's honk() method.

The Strategy implementation the simulator

```
public class DuckSimulator {
    public static void main(String[] args) {
        DuckSimulator simulator = new DuckSimulator();
        simulator.simulate();
    }
    void simulate() {
        Quackable mallardDuck = new MallardDuck();
        Quackable redheadDuck = new RedheadDuck();
        Quackable duckCall = new DuckCall();
        Quackable rubberDuck = new RubberDuck();
        Quackable gooseDuck = new GooseAdapter(new Goose());

        System.out.println("\nDuck Simulator: With Goose Adapter");

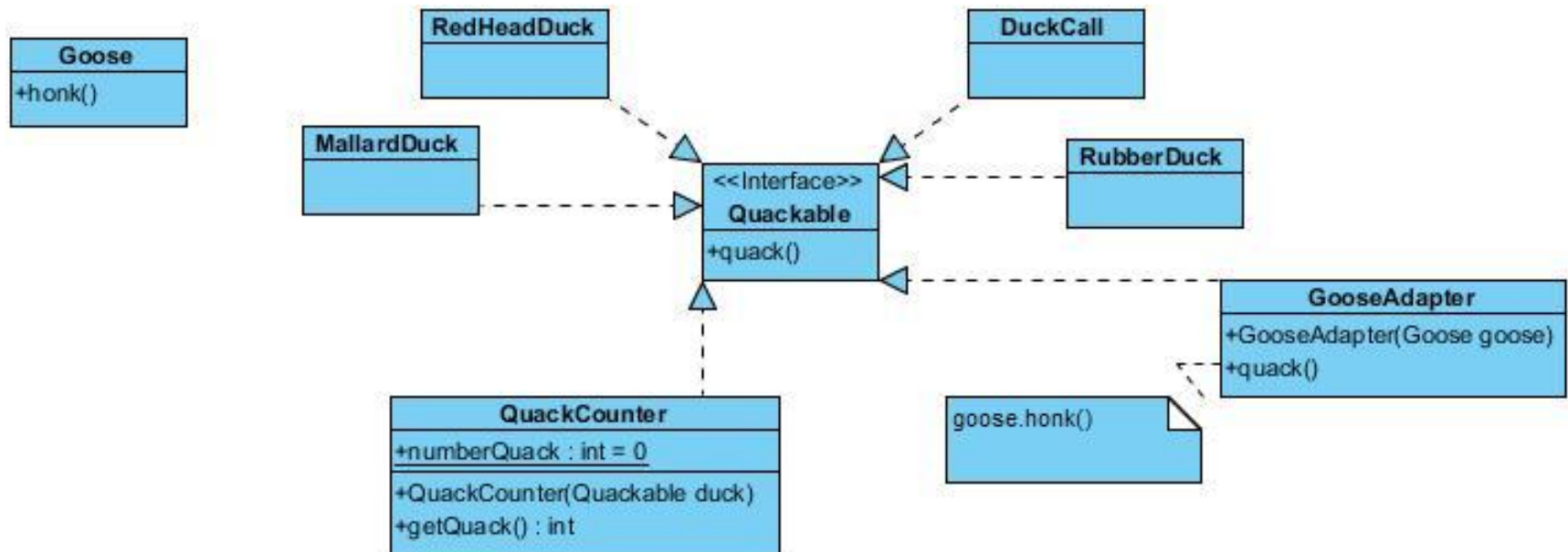
        simulate(mallardDuck);
        simulate(redheadDuck);
        simulate(duckCall);
        simulate(rubberDuck);
        simulate(gooseDuck);
    }

    void simulate(Quackable duck) {
        duck.quack();
    }
}
```

Issue

- ▶ We want to study is the total number of quacks made by a flock of ducks.
- ▶ How can we add the ability to count duck quacks without having to change the duck classes?
- ▶ Can you think of a pattern that would help?

Using a Decorator pattern



Using a Decorator pattern

- ▶ We can create a decorator that gives the ducks a new behavior (the behavior of counting) by wrapping them with a decorator object.

QuackCounter is
a decorator

```
public class QuackCounter implements Quackable {
```

```
    Quackable duck;
```

```
    static int numberOfQuacks;
```

we're counting ALL quacks,
so we'll use a static variable
to keep track.

```
    public QuackCounter (Quackable duck) {  
        this.duck = duck;  
    }
```

```
    public void quack() {  
        duck.quack();  
        numberOfQuacks++;  
    }
```

When quack() is called, we
delegate the call to the
Quackable we're decorating...

```
    public static int getQuacks() {  
        return numberOfQuacks;  
    }
```

We're adding one other method to the
decorator. This static method just returns the
number of quacks that have occurred in all
Quackables.

The Strategy implementation the simulator

```
public class DuckSimulator {
    public static void main(String[] args) {
        DuckSimulator simulator = new DuckSimulator();
        simulator.simulate();
    }
    void simulate() {
        Quackable mallardDuck = new QuackCounter(new MallardDuck());
        Quackable redheadDuck = new QuackCounter(new RedheadDuck());
        Quackable duckCall = new QuackCounter(new DuckCall());
        Quackable rubberDuck = new QuackCounter(new RubberDuck());
        Quackable gooseDuck = new GooseAdapter(new Goose());
        System.out.println("\nDuck Simulator: With Decorator");
        simulate(mallardDuck);
        simulate(redheadDuck);
        simulate(duckCall);
        simulate(rubberDuck);
        simulate(gooseDuck);
        System.out.println("The ducks quacked " +
                           QuackCounter.getQuacks() + " times");
    }
    void simulate(Quackable duck) {
        duck.quack();
    }
}
```

- ▶ You have to decorate objects to get decorated behavior.
 - ▶ that's the problem with wrapping objects: you have to make sure they get wrapped or they don't get the decorated behavior.
- ▶ Why don't we take the creation of ducks and localize it in one place;
 - ▶ in other words, let's take the duck creation and decorating and encapsulate it.
- ▶ What pattern does that sound like?

We need a factory

- ▶ We need some quality control to make sure our ducks get wrapped.
- ▶ We're going to build an entire factory just to produce them.
- ▶ The factory should produce a family of products that consists of different types of ducks, so we're going to use the Abstract Factory Pattern.

```
public abstract class AbstractDuckFactory {  
  
    public abstract Quackable createMallardDuck();  
    public abstract Quackable createRedheadDuck();  
    public abstract Quackable createDuckCall();  
    public abstract Quackable createRubberDuck();  
}
```


The abstract factory Class

```
public class DuckFactory extends AbstractDuckFactory {  
  
    public Quackable createMallardDuck() {  
        return new MallardDuck();  
    }  
  
    public Quackable createRedheadDuck() {  
        return new RedheadDuck();  
    }  
  
    public Quackable createDuckCall() {  
        return new DuckCall();  
    }  
  
    public Quackable createRubberDuck() {  
        return new RubberDuck();  
    }  
}
```

- ▶ This factory creates ducks without decorators ... how to solve the problem?

The factory

```
public class CountingDuckFactory extends AbstractDuckFactory {  
  
    public Quackable createMallardDuck() {  
        return new QuackCounter(new MallardDuck());  
    }  
  
    public Quackable createRedheadDuck() {  
        return new QuackCounter(new RedheadDuck());  
    }  
  
    public Quackable createDuckCall() {  
        return new QuackCounter(new DuckCall());  
    }  
  
    public Quackable createRubberDuck() {  
        return new QuackCounter(new RubberDuck());  
    }  
}
```

- ▶ Each method wraps the Quackable with the quack counting decorator.
- ▶ The simulator will never know the difference; it just gets back a Quackable.

The new simulator

```
public class DuckSimulator {  
    public static void main(String[] args) {  
        DuckSimulator simulator = new DuckSimulator();  
        AbstractDuckFactory duckFactory = new CountingDuckFactory();  
  
        simulator.simulate(duckFactory);  
    }  
    void simulate(AbstractDuckFactory duckFactory) {  
        Quackable mallardDuck = duckFactory.createMallardDuck();  
        Quackable redheadDuck = duckFactory.createRedheadDuck();  
        Quackable duckCall = duckFactory.createDuckCall();  
        Quackable rubberDuck = duckFactory.createRubberDuck();  
        Quackable gooseDuck = new GooseAdapter(new Goose());  
        System.out.println("\nDuck Simulator: With Abstract Factory");  
        simulate(mallardDuck);  
        simulate(redheadDuck);  
        simulate(duckCall);  
        simulate(rubberDuck);  
        simulate(gooseDuck);  
        System.out.println("The ducks quacked " +  
                           QuackCounter.getQuacks() + " times");  
    }  
    void simulate(Quackable duck) {  
        duck.quack();  
    }  
}
```

The simulate() method takes an AbstractDuckFactory and uses it to create ducks rather than instantiating them directly.

The same code in the rest of the class

Question

- ▶ We're still directly instantiating Geese by relying on concrete classes.
- ▶ How should it handle creating “goose ducks”?

Issue

- ▶ It's getting a little difficult to manage all these different ducks separately.
- ▶ Is there any way you can help us manage ducks as a whole, and perhaps even allow us to manage a few duck “families” that we'd like to keep track of?
- ▶ What we need is a way to talk about collections (flock) of ducks and even sub-collections of ducks.
 - ▶ It would also be nice if we could apply operations across the whole set of ducks.
- ▶ What pattern can help us?

Creating flocks of ducks

- ▶ The Composite pattern allows us to treat a collection of objects (a flock) in the same way as individual objects (a duck).

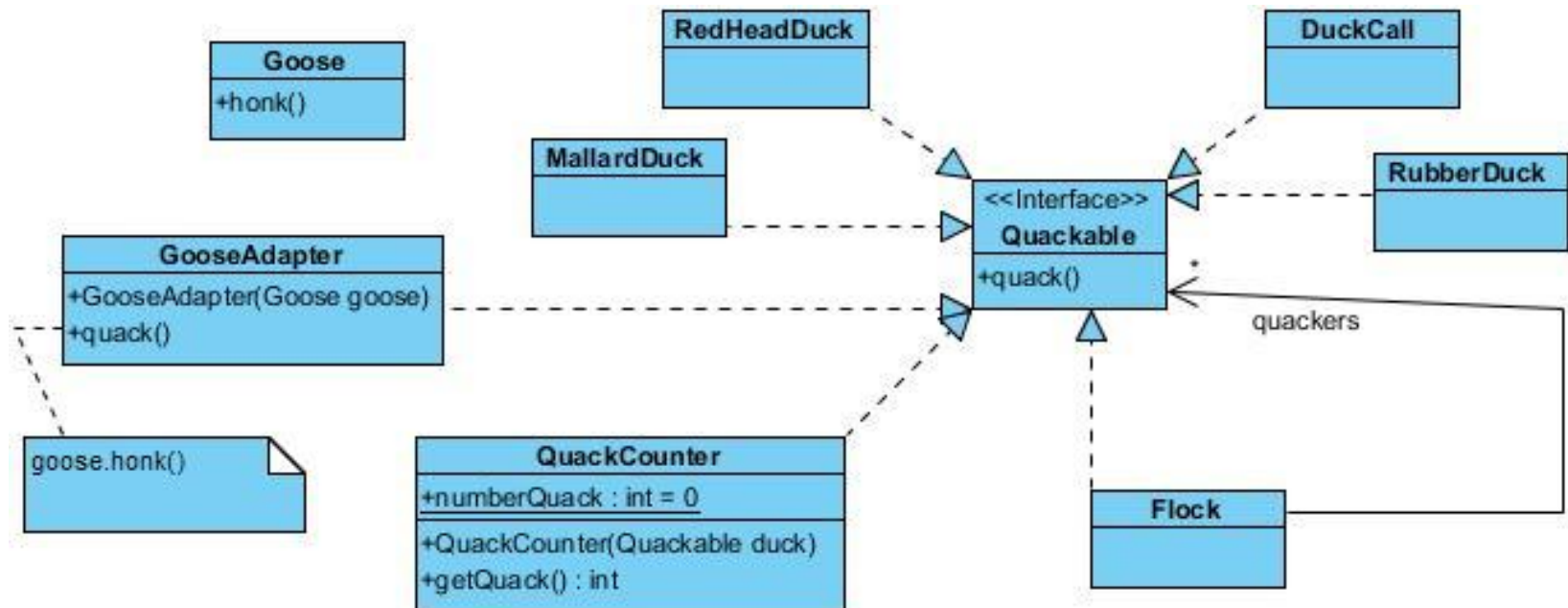
```
public class Flock implements Quackable {
    ArrayList<Quackable> quackers = new ArrayList<Quackable>();

    public void add(Quackable quacker) {
        quackers.add(quacker);
    }

    public void quack() {
        Iterator<Quackable> iterator = quackers.iterator();
        while (iterator.hasNext()) {
            Quackable quacker = iterator.next();
            quacker.quack();
        }
    }
}
```

We have used also an iterator!

Creating flocks of ducks



- ▶ We don't have to alter the code ... only to add flocks!

The simulator

```
public class DuckSimulator {  
    // main method here  
    void simulate(AbstractDuckFactory duckFactory) {  
        Quackable redheadDuck = duckFactory.createRedheadDuck();  
        Quackable rubberDuck = duckFactory.createRubberDuck();  
        Quackable gooseDuck = new GooseAdapter(new Goose());  
        System.out.println("\nDuck Simulator: With Composite - Flocks");  
        Flock flockOfDucks = new Flock();  
        flockOfDucks.add(redheadDuck);  
        flockOfDucks.add(rubberDuck);  
        flockOfDucks.add(gooseDuck);  
        Flock flockOfMallards = new Flock();  
        Quackable mallardOne = duckFactory.createMallardDuck();  
        Quackable mallardTwo = duckFactory.createMallardDuck();  
        flockOfMallards.add(mallardOne);  
        flockOfMallards.add(mallardTwo);  
        flockOfDucks.add(flockOfMallards);  
        System.out.println("\nDuck Simulator: Whole Flock Simulation");  
        simulate(flockOfDucks);  
        System.out.println("\nDuck Simulator: Mallard Flock Simulation");  
        simulate(flockOfMallards);  
        System.out.println("\nThe ducks quacked " +  
                           QuackCounter.getQuacks() + " times");  
    }  
    void simulate(Quackable duck) {  
        duck.quack();  
    }  
}
```



Safety versus transparency

- ▶ You might remember that in the Composite Pattern presented the composites (the Menus) and the leaf nodes (the MenuItem)s had the same exact set of methods, including the add() method.
 - ▶ Because they had the same set of methods, we could call methods on MenuItem)s that didn't really make sense.
- ▶ The benefit of this was that the distinction between leaves and composites was transparent: the client didn't have to know whether it was dealing with a leaf or a composite; it just called the same methods on both.
- ▶ Here, we've decided to keep the composite's child maintenance methods separate from the leaf nodes: that is, only Flocks have the add() method.
 - ▶ We know it doesn't make sense to try to add something to a Duck, and in this implementation, you can't. You can only add() to a Flock.
- ▶ So this design is safer: you can't call methods that don't make sense on components,
 - ▶ but it's less transparent.
 - ▶ Now the client has to know that a Quackable is a Flock in order to add Quackables to it.
- ▶ As always, there are trade-offs when you do OO design and you need to consider them as you create your own composites.

Issue

- ▶ Now we have the opposite request: we also need to track individual ducks.
- ▶ Can we design a way to keep track of individual duck quacking in real time?

The observer pattern

- ▶ First we need an Observable interface.
 - ▶ An Observable needs methods for registering and notifying observers.
 - ▶ We could also have a method for removing observers, but we'll keep the implementation simple here and leave that out.

```
public interface QuackObservable {  
    public void registerObserver(Observer observer);  
    public void notifyObservers();  
}
```

```
public interface Quackable extends QuackObservable {  
    public void quack();  
}
```

The observable

- ▶ We need to make sure all the concrete classes that implement Quackable can handle being a QuackObservable.
- ▶ We could approach this by implementing registration and notification in each and every class.
- ▶ However, in this implementation, we will encapsulate the registration and notification code in another class, call it Observable, and compose it with a QuackObservable.
- ▶ That way, we only write the real code once and the QuackObservable just needs enough code to delegate to the helper class Observable.

The Observable Helper Class

```
public class Observable implements QuackObservable {
    ArrayList<Observer> observers = new ArrayList<Observer>();
    QuackObservable duck;

    public Observable(QuackObservable duck) {
        this.duck = duck;
    }

    public void registerObserver(Observer observer) {
        observers.add(observer);
    }

    public void notifyObservers() {
        Iterator<Observer> iterator = observers.iterator();
        while (iterator.hasNext()) {
            Observer observer = (Observer) iterator.next();
            observer.update(duck);
        }
    }
}
```

The new Concrete Classes of Ducks

```
public class MallardDuck implements Quackable {
    Observable observable;

    public MallardDuck() {
        observable = new Observable(this);
    }

    public void quack() {
        System.out.println("Quack");
        notifyObservers();
    }

    public void registerObserver(Observer observer) {
        observable.registerObserver(observer);
    }

    public void notifyObservers() {
        observable.notifyObservers();
    }
}
```

We have to change the implementation of QuackCounter , and the other kinds of duck too.



The observer

```
public interface Observer {  
    public void update(QuackObservable duck);  
}  
  
public class DuckObserver implements Observer {  
  
    public void update(QuackObservable duck) {  
        System.out.println(" DuckObserver : " + duck + " just quacked.");  
    }  
}
```

The Simulator

```
public class DuckSimulator {
    public static void main(String[] args) {
        DuckSimulator simulator = new DuckSimulator();
        AbstractDuckFactory duckFactory = new CountingDuckFactory();
        simulator.simulate(duckFactory);
    }
    void simulate(AbstractDuckFactory duckFactory) {
        Quackable mallardDuck = duckFactory.createMallardDuck();
        Quackable redheadDuck = duckFactory.createRedheadDuck();
        Quackable duckCall = duckFactory.createDuckCall();
        Quackable rubberDuck = duckFactory.createRubberDuck();
        System.out.println("\nDuck Simulator: With Abstract Factory");
        System.out.println("The ducks quacked " + QuackCounter.getQuacks() + "
times");
        System.out.println("\nDuck Simulator: With Observer");
        DuckObserver duckObserver = new DuckObserver();
        mallardDuck.registerObserver(duckObserver);
        simulate(mallardDuck);
        System.out.println("\nThe ducks quacked " +
            QuackCounter.getQuacks() + " times");
    }
    void simulate(Quackable duck) {
        duck.quack();
    }
}
```



Missing part ...

- ▶ What if we want to observe an entire flock?
- ▶ What does that mean anyway?
- ▶ Think about it like this: if we observe a composite, then we're observing everything in the composite.
- ▶ So, when you register with a flock, the flock composite should make sure you get registered with all its children, which may include other flocks.

the new Flock

```
public class Flock implements Quackable {
    Observable observable;
    ArrayList<Quackable> quackers = new ArrayList<Quackable>();
    public void add(Quackable quacker) {
        quackers.add(quacker);
    }

    public void quack() {
        Iterator<Quackable> iterator = quackers.iterator();
        while (iterator.hasNext()) {
            Quackable quacker = iterator.next();
            quacker.quack();
        }
    }

    public void registerObserver(Observer observer) {
        Iterator<Quackable> iterator = quackers.iterator();
        while (iterator.hasNext()) {
            Quackable quacker = iterator.next();
            quacker.registerObserver(observer);
        }
    }

    public void notifyObservers() {
        observable.notifyObservers();
    }
}
```

If we observe a flock, we observe all the components of a flock



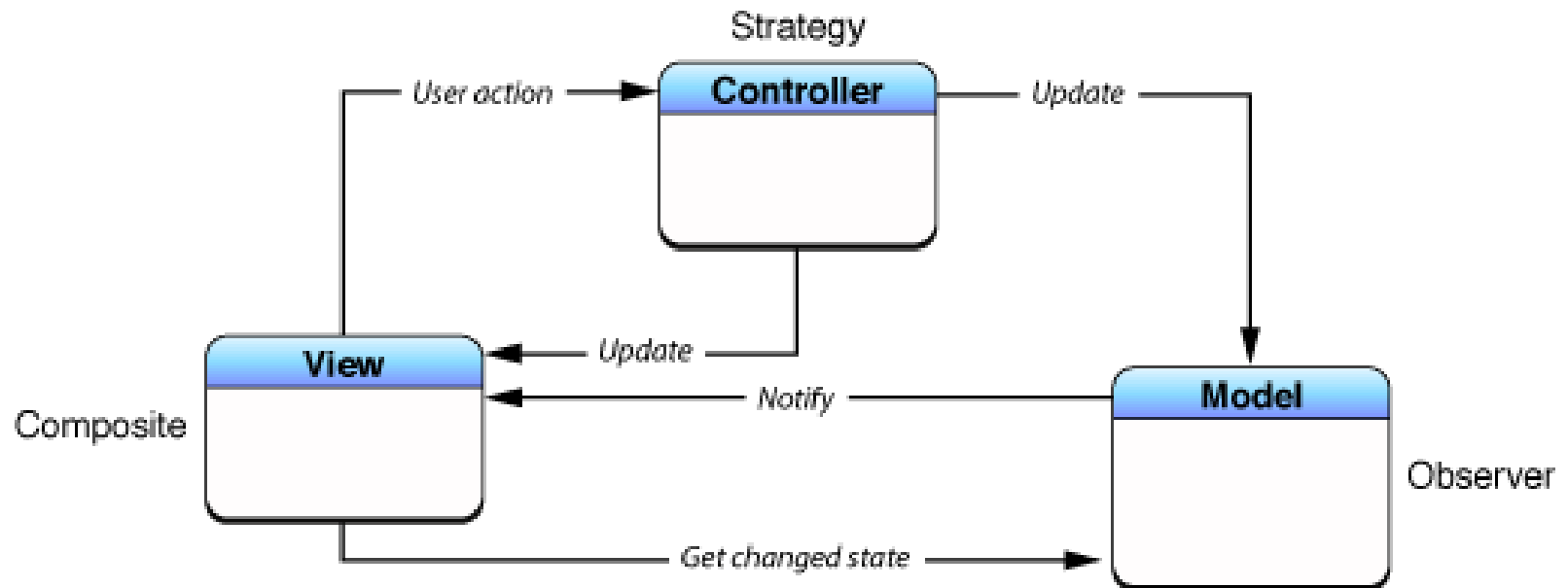
```
public class DuckSimulator {
    public static void main(String[] args) {
        DuckSimulator simulator = new DuckSimulator();
        AbstractDuckFactory duckFactory =
            new CountingDuckFactory();
        simulator.simulate(duckFactory);
    }
    void simulate(AbstractDuckFactory duckFactory) {
        Quackable mallardDuck = duckFactory.createMallardDuck();
        Quackable redheadDuck = duckFactory.createRedheadDuck();
        Quackable duckCall = duckFactory.createDuckCall();
        Quackable rubberDuck = duckFactory.createRubberDuck();
        System.out.println("\nDuck Simulator: With Abstract Factory");
        Flock flockOfMallards = new Flock();
        flockOfMallards.add(mallardDuck);
        flockOfMallards.add(duckCall);
        flockOfMallards.add(rubberDuck);
        flockOfMallards.add(redheadDuck);
        System.out.println("The ducks quacked " + QuackCounter.getQuacks() +
            " times");
        System.out.println("\nDuck Simulator: With Observer");
        DuckObserver duckObserver = new DuckObserver();
        flockOfMallards.registerObserver(duckObserver);
        simulate(flockOfMallards);
        System.out.println("\nThe ducks quacked " + QuackCounter.getQuacks()
            + " times");
    }
    void simulate(Quackable duck) {duck.quack();
    }
```

Observing a Flock

Compound Patterns

-example

- ▶ MVC can be considered as a compound pattern as depicted in this picture



References

- ▶ Eric Freeman, Elisabeth Robson, Bert Bates, Kathy Sierra: Head First Design Patterns, O'Reilly Media 2004
- ▶ Erich Gamma, Ralph Johnson, Richard Helm, John Vlissides: Design Patterns: Elements of Reusable Object-Oriented Software, Addison-Wesley, 1994

MATERIALE DIDATTICO INTERNO DEL CORSO DI Ingegneria del Software.

La diffusione e pubblicazione on line del materiale è assolutamente proibita.